Kommunikationstechnik Teil 4 MQTT

DI (FH) Andreas Pötscher

HTL Litec

Einleitung MQTT

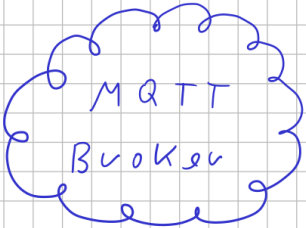
LINZER TECHNIKUM
HTL Paul-Hahn-Straße

MQTT (Message Queuing Telemetry Transport) ist ein schlankes Messaging-Protokoll, das speziell für mobile Anwendungen und das Internet der Dinge (IoT) entwickelt wurde. Neben Zuverlässigkeit lag der Fokus bei der Entwicklung vor allem auf einem sparsamen Umgang mit Bandbreite und Systemressourcen. Das Protokoll ist einfach aufgebaut und sehr leichtgewichtig.

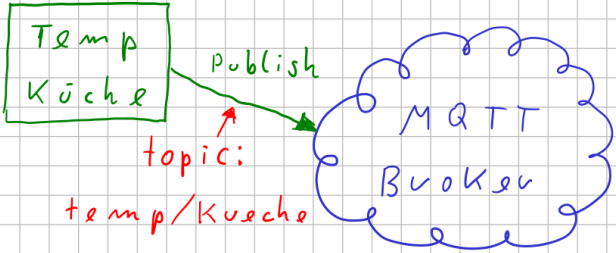
Publish Subscribe Modell

Beim Pub/Sub-Modell entfällt die direkte Kommunikation zwischen dem Sender und dem Empfänger einer Nachricht. Stattdessen ermöglicht der Einsatz eines Brokers (eine Datenvermittlungssoftware) die Kontrolle darüber, welcher Empfänger welche Nachricht erhält. Der Broker ist eine eigenständige Software, die typischerweise auf einem separaten Rechner (z. B. in der Cloud) gehostet wird und die Verteilung der Daten übernimmt.

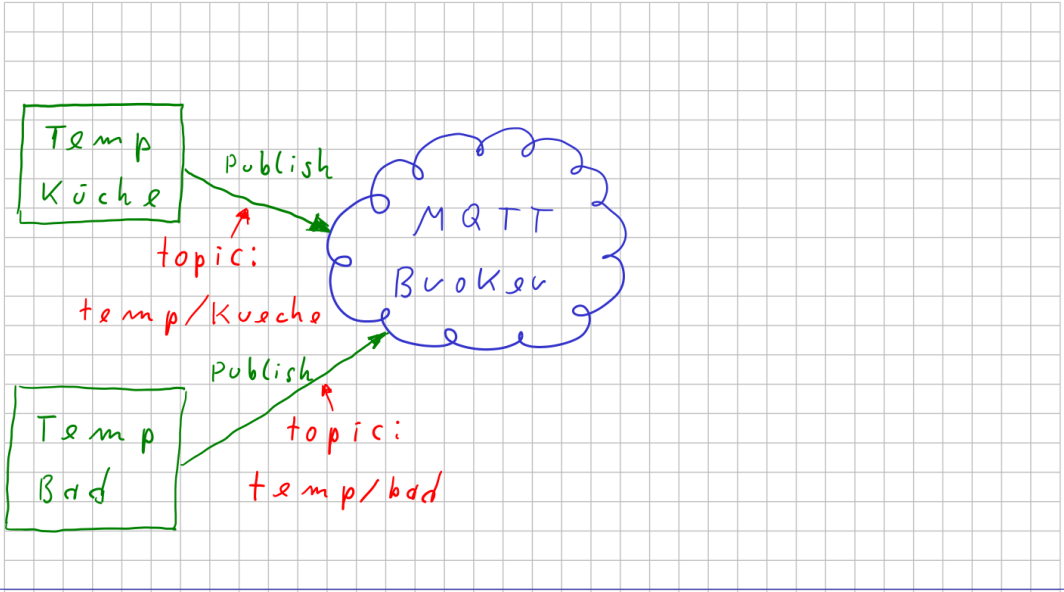
Publish Subscribe Modell



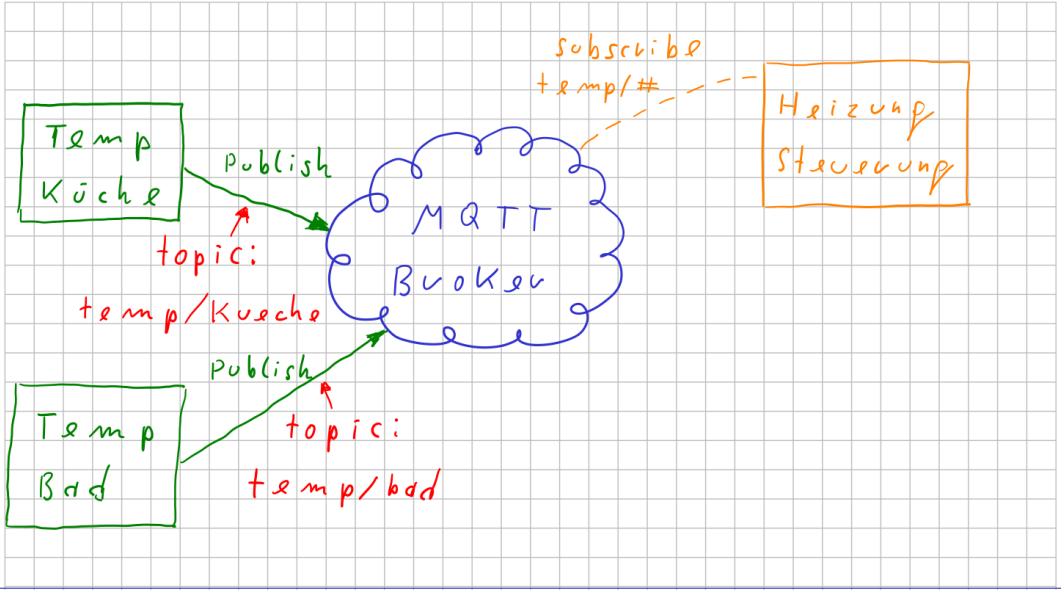
Publish Subscribe Modell



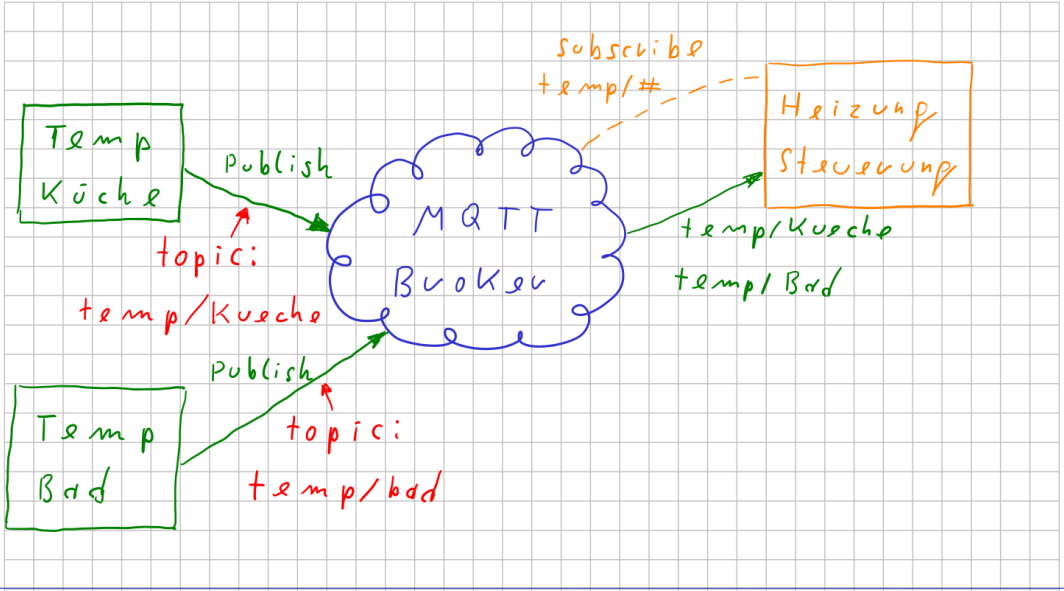
Publish Subscribe Modell



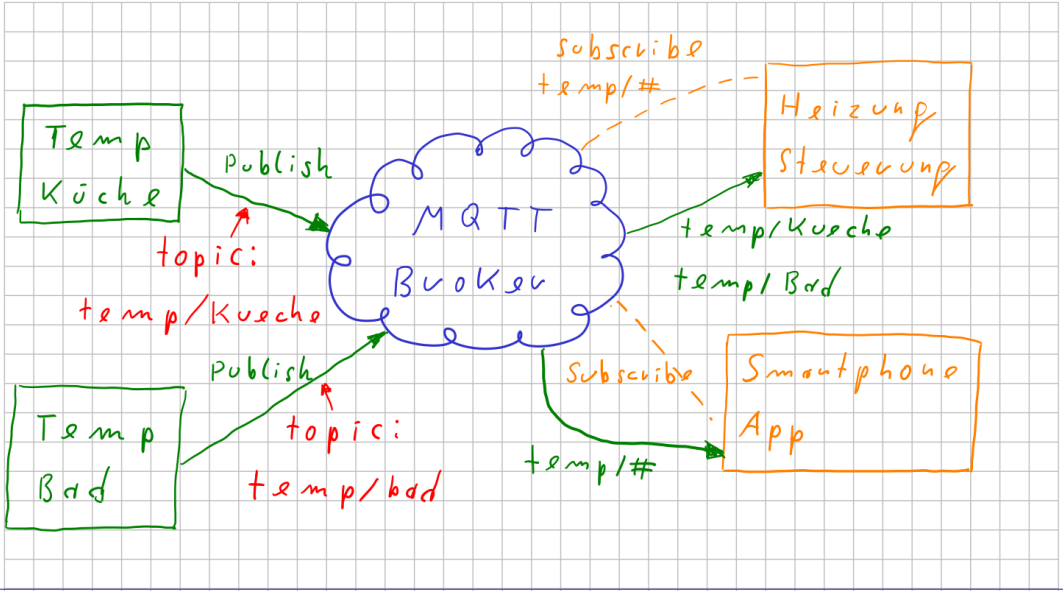
Publish Subscribe Modell



Publish Subscribe Modell



Publish Subscribe Modell



Publish Subscribe Modell

Die Entkopplung von Sender und Empfänger bietet zwei wesentliche Vorteile: die Trennung von Raum und Zeit.

- **Raum:** Der Sender und der Empfänger müssen über das Netzwerk nicht gegenseitig erreichbar sein. Sie können sich beispielsweise in zwei privaten Netzwerken befinden und über einen öffentlich erreichbaren Broker kommunizieren.
- **Zeit:** Sender und Empfänger müssen nicht zur selben Zeit laufen. Die Daten werden vom Broker zwischengespeichert und zeitversetzt zugestellt. Es gehen also keine Daten verloren. Selbst wenn der Empfänger für längere Zeit offline ist.

Publish Subscribe Modell

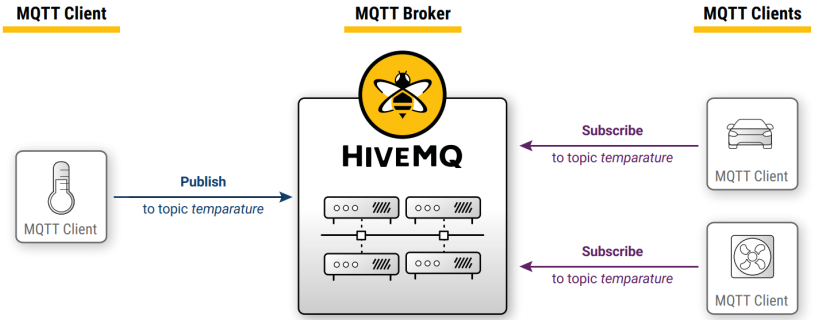


Figure 7: Publish Subscribe Modell bei MQTT (Quelle: HiveMQ)

MQTT Client

LINZER TECHNIKUM
HTL Paul-Hahn-Straße

Ein MQTT-Client ist jedes netzwerkfähige Gerät, das über MQTT kommuniziert. Clients können gleichzeitig Nachrichten veröffentlichen und abonnieren. Sie können, müssen aber nicht, gleichzeitig Publisher und Subscriber sein.

MQTT Broker

LINZER TECHNIKUM
HTL Paul-Hahn-Straße

Ein MQTT-Broker ist die zentrale Instanz innerhalb des Publish/Subscribe-Kommunikationsmodells. Er übernimmt eine Vielzahl kritischer Aufgaben, um den Nachrichtenaustausch zwischen MQTT-Clients effizient, zuverlässig und sicher zu ermöglichen. Zu den Hauptaufgaben eines Brokers gehört der Empfang aller veröffentlichten Nachrichten von Clients. Anschließend filtert der Broker diese Nachrichten basierend auf den jeweiligen Topics und ermittelt, welche Clients ein entsprechendes Abonnement besitzen. Nur an diese abonnierenden Clients werden die Nachrichten weitergeleitet.

Ausprobieren

Installieren Sie die Software MQTT Explorer

- https://mqtt-explorer.com/
- Windows installer

Verbinden Sie sich mit dem Broker

`mqtt://test.mosquitto.org/ auf Port 1883`

Verbindungsaufbau

Der Verbindungsaufbau von Client zum Broker wird vom Client initiiert. Dazu sendet der Client eine MQTT Connect Message zum Broker. Diese beinhaltet folgende Parameter:

- **ClientID** Eindeutige Kennung des Client. Wird meistens vom Client automatisch generiert.

Verbindungsaufbau

Der Verbindungsaufbau von Client zum Broker wird vom Client initiiert. Dazu sendet der Client eine MQTT Connect Message zum Broker. Diese beinhaltet folgende Parameter:

- **ClientID** Eindeutige Kennung des Client. Wird meistens vom Client automatisch generiert.
- **Clean Session**

Verbindungsaufbau

LINZER TECHNIKUM
HTL Paul-Hahn-Straße

Der Verbindungsaufbau von Client zum Broker wird vom Client initiiert. Dazu sendet der Client eine MQTT Connect Message zum Broker. Diese beinhaltet folgende Parameter:

- **ClientID** Eindeutige Kennung des Client. Wird meistens vom Client automatisch generiert.
- **Clean Session**
- `true` bedeutet, dass der Broker beim Verbindungsabbruch die Sitzungsdaten des Clients löscht. Das heißt, wenn sich derselbe Client erneut verbindet, wird er vom Broker nicht wiedererkannt.

Verbindungsaufbau

Der Verbindungsaufbau von Client zum Broker wird vom Client initiiert. Dazu sendet der Client eine MQTT Connect Message zum Broker. Diese beinhaltet folgende Parameter:

- **ClientID** Eindeutige Kennung des Client. Wird meistens vom Client automatisch generiert.
- **Clean Session**
- `true` bedeutet, dass der Broker beim Verbindungsabbruch die Sitzungsdaten des Clients löscht. Das heißt, wenn sich derselbe Client erneut verbindet, wird er vom Broker nicht wiedererkannt.
- `false` bedeutet, dass der Broker die Verbindungsdaten des Clients speichert. Wenn sich derselbe Client erneut verbindet, wird er wiedererkannt. Das bedeutet, dass bestehende Subscriptions erhalten bleiben. Falls der Client während seiner "Offline-Zeit" Nachrichten verpasst hat, werden diese – je nach QoS – nachträglich zugestellt.

Verbindungsaufbau

LINZER TECHNIKUM
HTL Paul-Hahn-Straße

- **Username/Passwort** Für die Authentifizierung des Clients.

Verbindungsaufbau

LINZER TECHNIKUM
HTL Paul-Hahn-Straße

- **Username/Passwort** Für die Authentifizierung des Clients.
- **Will Message** Messages die vom Broker im Namen des Clients gesendet wird wenn die Verbindung unerwartet unterbrochen wird.

Verbindungsaufbau

LINZER TECHNIKUM
HTL Paul-Hahn-Straße

- **Username/Passwort** Für die Authentifizierung des Clients.
- **Will Message** Messages die vom Broker im Namen des Clients gesendet wird wenn die Verbindung unerwartet unterbrochen wird.
- **KeepAlive** Zeitintervall in Sekunden in dem zwischen Client und Broker mindestens ein KeepAlive Signal gesendet werden muss. Das Keepalive Signal wird in Form eines Pings gesendet.

Publish

Hat ein Client die Verbindung zum Broker aufgebaut kann er dort Daten publishen. Eine Publish Message enthält folgende Informationen:

- **Topic:** Das Topic ist ein einfacher String der das Thema der Nachricht enthält. Das Topic kann durch slashes strukturiert werden. Z.B. `home/livingroom/temperature`.

Publish

Hat ein Client die Verbindung zum Broker aufgebaut kann er dort Daten publishen. Eine Publish Message enthält folgende Informationen:

- **Topic:** Das Topic ist ein einfacher String der das Thema der Nachricht enthält. Das Topic kann durch slashes strukturiert werden. Z.B. `home/livingroom/temperature`.
- **QoS** Quality of Service. Es gibt 3 Levels 0,1,2. Bestimmt mit welcher Zuverlässigkeit die Nachrichten ankommen.

Publish

Hat ein Client die Verbindung zum Broker aufgebaut kann er dort Daten publishen. Eine Publish Message enthält folgende Informationen:

- **Topic:** Das Topic ist ein einfacher String der das Thema der Nachricht enthält. Das Topic kann durch slashes strukturiert werden. Z.B. `home/livingroom/temperature`.
- **QoS** Quality of Service. Es gibt 3 Levels 0,1,2. Bestimmt mit welcher Zuverlässigkeit die Nachrichten ankommen.
- **Retain Flag** Bestimmt ob eine Message beim Broker als letzte bekannte Message des Clients zu einem bestimmten Topic gespeichert werden soll.

Publish

Hat ein Client die Verbindung zum Broker aufgebaut kann er dort Daten publishen. Eine Publish Message enthält folgende Informationen:

- **Topic:** Das Topic ist ein einfacher String der das Thema der Nachricht enthält. Das Topic kann durch slashes strukturiert werden. Z.B. `home/livingroom/temperature`.
- **QoS** Quality of Service. Es gibt 3 Levels 0,1,2. Bestimmt mit welcher Zuverlässigkeit die Nachrichten ankommen.
- **Retain Flag** Bestimmt ob eine Message beim Broker als letzte bekannte Message des Clients zu einem bestimmten Topic gespeichert werden soll.
- **Payload** Inhalt der Nachricht. Unterschiedliche Datentypen sind hier möglich. Zahlenwert, Strings, verschachtelte Daten als json Sting. Alle Arten von Binärdaten.

Publish

Hat ein Client die Verbindung zum Broker aufgebaut kann er dort Daten publishen. Eine Publish Message enthält folgende Informationen:

- **Topic:** Das Topic ist ein einfacher String der das Thema der Nachricht enthält. Das Topic kann durch slashes strukturiert werden. Z.B. `home/livingroom/temperature`.
- **QoS** Quality of Service. Es gibt 3 Levels 0,1,2. Bestimmt mit welcher Zuverlässigkeit die Nachrichten ankommen.
- **Retain Flag** Bestimmt ob eine Message beim Broker als letzte bekannte Message des Clients zu einem bestimmten Topic gespeichert werden soll.
- **Payload** Inhalt der Nachricht. Unterschiedliche Datentypen sind hier möglich. Zahlenwert, Strings, verschachtelte Daten als json Sting. Alle Arten von Binärdaten.
- **Packet Identifier** Eindeutige Kennung der Nachricht. Wichtig für QoS > 0.

Topics

Topics sind Adressinformationen, über die Nachrichten an bestimmte Subscriber zugestellt werden können. Der Datentyp eines Topics ist immer ein String. Ein Topic besteht aus einem oder mehreren Topic-Levels, die durch einen Schrägstrich (/) getrennt sind, zum Beispiel:

```
myhome/groundfloor/livingroom/temperature
```

Multi Level Wildcard

Mit einer Multi-Level-Wildcard können alle unter einem bestimmten Topic-Level liegenden Topics abonniert werden. Die Multi-Level-Wildcard darf nur am Ende eines Subscriptionspfads verwendet werden. Zum Beispiel mit:

```
myhome/groundfloor/#
```

werden folgende Topics abonniert:

```
myhome/groundfloor/livingroom/temperature
myhome/groundfloor/livingroom/brightness
myhome/groundfloor/kitchen/temperature
```

Single Level Wildcard +

Mit einer Single-Level-Wildcard kann auf alle Möglichkeiten eines einzelnen Topic-Levels abonniert werden. Zum Beispiel mit:

`myhome/groundfloor/+/temperature`

werden folgende Topics abonniert:

```
myhome/groundfloor/livingroom/temperature
myhome/groundfloor/kitchen/temperature
myhome/groundfloor/bathroom/temperature
```

Quality of Service

LINZER TECHNIKUM
HTL Paul-Hahn-Straße

Der Begriff Quality of Service (QoS) bezeichnet im MQTT-Protokoll den Grad der Zuverlässigkeit, mit dem Nachrichten zwischen Publisher und Subscriber übertragen werden. MQTT definiert drei QoS-Stufen: 0, 1 und 2. Jede Stufe bietet spezifische Garantien hinsichtlich der Nachrichtenübermittlung und stellt einen Kompromiss zwischen Zuverlässigkeit und Ressourcenverbrauch dar.

Die QoS wird für beide Seiten der Datenübertragung seperat eingestellt:

- Für die Übertragung vom Client (Publisher) zum Broker
- Für die Übertragung vom Broker zum Client (Subscriber)

QoS 0: At most once

QoS 0 ist die einfachste und schnellste Stufe der Nachrichtenübertragung im MQTT-Protokoll. Dabei wird die Nachricht ohne jegliche Bestätigung vom Sender an den Broker übertragen. Es gibt keine Garantie, dass die Nachricht den Empfänger erreicht. Weder der Sender noch der Broker speichern die Nachricht oder versuchen eine erneute Übertragung. Daher wird dieses Verfahren auch als „Fire and Forget“ bezeichnet.

Figure 8: QoS 0 (Quelle: HiveMQ)

QoS 1: At least once

Bei QoS 1 wird garantiert, dass eine Nachricht mindestens einmal zugestellt wird. Der Sender speichert die Nachricht so lange, bis er eine Bestätigung (PUBACK) vom Empfänger erhalten hat. Erfolgt diese Bestätigung nicht innerhalb eines festgelegten Timeouts, wird die Nachricht erneut gesendet.

Das bedeutet, dass beim Einsatz von QoS 1 Duplikate möglich sind, die vom Empfänger entsprechend behandelt (z. B. gefiltert) werden müssen.

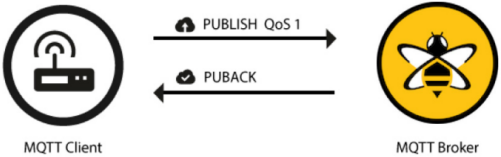


Figure 9: QoS 1 (Quelle: HiveMQ)

QoS 2: Exactly once

Bei QoS 2 wird die Nachricht garantiert genau einmal zugestellt. Es handelt sich um die sicherste, aber zugleich auch langsamste Nachrichtenqualität im MQTT-Protokoll. Für die Zustellung sind zwei Handshakes, also insgesamt vier Nachrichten, erforderlich: Ein erster Handshake zur Übertragung und Bestätigung der Nachricht, und ein zweiter, um die Packet Identifier (Packet ID) beim Sender und Empfänger freizugeben, damit diese erneut verwendet werden können.

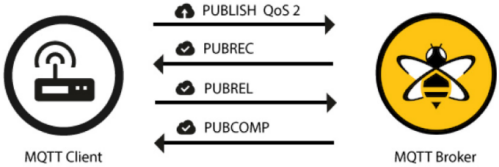


Figure 10: QoS 2 (Quelle: HiveMQ)

Retained Messages

Wird bei einer Nachricht das Retained-Flag gesetzt, speichert der Broker die letzte Nachricht zu diesem Topic zwischen. Jeder Client, der dieses Topic abonniert, erhält beim Abonnement diese Nachricht als zuletzt bekannten gültigen Wert sofort zugestellt.